JUST EAT · ANDROID PROJECT

后续开发指南
与团队规则

面向后续三人协作的统一手册：说明功能路线、轻量 MVVM 边界、Firebase 数据与权限方向、页面实现方式、测试发布流程，以及所有成员必须遵守的交付规则。

项目	Just Eat Android 课程小组项目
包名	com.groupproject.justeat
技术栈	Java 17 · XML · ViewBinding · MVVM · Glide · Firebase-ready
当前范围	LoginActivity 界面、输入校验与 ViewModel 状态；其余功能为后续规划
版本	1.0 · 2026-09-05

重要说明：本手册中的 Firebase 数据结构、权限矩阵和后续页面均为开发约定与建议，不代表这些功能已经实现。

阅读导航

建议新成员先阅读开发指南第 3–8 节理解代码与数据边界，再阅读团队规则第 2–11 节完成第一次提交。每次新增功能前，使用文末检查表确认设计；提交前，使用 Definition of Done 确认交付。

01	开发指南	产品角色、MVVM 架构、功能路线、Firebase 数据、测试与发布
02	团队规则	Git、Java、UI、状态、数据、安全、审查和完成标准

当前项目边界

当前代码提供 Java/XML 单 app 模块基础框架、BaseActivity、BaseViewModel、BaseDialog、ViewBinding、Glide 封装、实体类、Repository 接口，以及 LoginActivity 的界面、输入校验和状态管理。尚未添加 google-services.json，也没有执行真实登录。

使用方式

- 做功能前：检查职责包、数据字段、状态值和 Firebase 权限方向。
- 开发中：保持 Activity 轻量，让 ViewModel 与 Repository 承担业务和数据访问。
- 提交前：运行单元测试、Lint、Debug 构建和相关页面冒烟测试。
- 合并前：完成至少一名成员审查，并同步更新相关文档与规则。

PART 01

Just Eat 后续开发指南

文档版本：1.0
适用项目：com.groupproject.justeat
技术栈：Java 17、XML、ViewBinding、AndroidX Lifecycle、Glide、Firebase

1. 文档目的

本文档说明当前基础框架之后应如何继续开发 Just Eat。它用于统一三人小组对 架构、功能顺序、数据模型、Firebase 接入、测试和交付方式的理解。

当前项目只完成 LoginActivity 的界面、本地输入校验和 ViewModel 状态管理。 登录成功提示不代表真实认证成功；真实 Firebase Authentication、主页、菜单、 预订、订单、员工和经理页面均属于后续工作。

2. 产品角色与目标功能

2.1 Customer

- 注册、登录、退出和维护个人资料。
- 查看菜单分类、菜品详情、图片、价格和供应状态。
- 选择餐桌和时间并创建、查看或取消预订。
- 创建订单、添加菜品备注并查看订单进度。

2.2 Staff

- 查看和处理餐桌预订。
- 更新餐桌可用状态。
- 接收订单并更新制作、就绪和完成状态。
- 维护自己的个人资料。

2.3 Manager

- 新增、修改、下架菜单分类与菜品。
- 新增、修改或停用餐桌。
- 创建、启用、停用员工账号。
- 查看餐厅级预订和订单状态。

3. 当前技术基线

- 单一 app 模块，应用 ID 和 namespace 均为 com.groupproject.justeat。
- Java 17；不使用 Kotlin 或 Jetpack Compose。
- XML 布局配合 ViewBinding；不使用 DataBinding。

- 最低 API 24，compile SDK 和 target SDK 为 35。
- ViewModel 和 LiveData 管理页面状态。
- Glide 负责本地或远程图片加载。
- Firebase Authentication 与 Realtime Database 依赖已经声明。
- `google-services.json` 尚未加入，并已被 Git 忽略。

4. 架构与数据流

项目使用轻量 MVVM。依赖方向固定如下：

```
Activity / Dialog
    ↓ 用户操作、观察状态
ViewModel + UiState + Validator
    ↓ 调用抽象接口
Repository interface
    ↓ 后续实现
Firebase Authentication / Realtime Database
```

反向依赖不允许出现：Repository 不引用 Activity，ViewModel 不引用 ViewBinding， 实体类不引用 Android UI 类型。

4.1 包职责

```
base/                Activity、Dialog、ViewModel 的通用生命周期能力
data/model/          Firebase 可反序列化的 Java 实体
data/repository/     上层依赖的数据访问接口
data/repository/firebase/  后续 Firebase 接口实现
domain/constant/     Firebase 中保存的角色和状态字符串
feature/login/       登录功能
feature/customer/    后续顾客功能
feature/staff/       后续员工功能
feature/manager/     后续经理功能
ui/dialog/           跨功能复用的对话框
util/                无业务状态的通用工具
```

5. 后续功能建议顺序

阶段 1：Firebase 与认证

1. 按 `docs/FIREBASE_SETUP.md` 添加自己的 Firebase 项目。
2. 在 `data/repository/firebase` 实现 `AuthRepository`。
3. 保留现有本地校验，在校验通过后才发起网络登录。
4. 登录成功后读取 `users/{uid}` 的角色和账号状态。
5. 根据 `UserRole` 进入 Customer、Staff 或 Manager 的主页。
6. 增加注册和忘记密码页面。

阶段 2：Customer 主流程

1. 菜单分类和菜品列表。
2. 菜品详情与购物车。
3. 餐桌列表和预订创建。
4. 订单提交与状态查看。

5. 个人资料编辑。

阶段 3：Staff 主流程

1. 待处理预订列表与状态更新。
2. 餐桌占用、预订和不可用状态管理。
3. 订单接收、制作、就绪和完成流程。

阶段 4：Manager 主流程

1. 菜单分类和菜品 CRUD。
2. 餐桌 CRUD。
3. 员工账号与状态管理。
4. 全局订单和预订查看。

阶段 5：质量与交付

1. 完整错误状态、空状态和离线提示。
2. 关键流程自动化测试。
3. Firebase 安全规则和索引测试。
4. 无障碍、横竖屏和小屏适配。
5. Release 构建、演示数据和课程展示材料。

6. 新页面开发方式

每个功能放入自己的 `feature/<role-or-feature>` 包。一个普通页面建议包含：

```
feature/customer/menu/
├── MenuActivity.java
├── MenuViewModel.java
└── MenuUiState.java
```

存在复杂输入规则时再增加 `MenuValidator.java`。只有多个页面共享同一 UI 组件时，才把组件移入 `ui`；不要提前建立没有实际使用者的抽象层。

6.1 Activity

- 继承 `BaseActivity<对应的Binding>`。
- 只负责绑定点击、读取输入、观察状态和渲染界面。
- 不直接调用 Firebase SDK，不计算业务规则。
- 不保存密码、Token 或大型数据列表。

示例：

```
public class MenuActivity extends BaseActivity<ActivityMenuBinding> {
    private MenuViewModel viewModel;

    @Override
    protected ActivityMenuBinding inflateBinding(LayoutInflater inflater) {
        return ActivityMenuBinding.inflate(inflater);
    }

    @Override
    protected void onCreate(Bundle savedInstanceState) {
        super.onCreate(savedInstanceState);
        viewModel = new ViewModelProvider(this).get(MenuViewModel.class);
        viewModel.getUiState().observe(this, this::render);
    }

    private void render(MenuUiState state) {
        // 只根据 state 更新 ViewBinding。
    }
}
```

6.2 ViewModel 与 UiState

- ViewModel 负责业务流程、调用 Repository 并发布不可变 UiState。
- ViewModel 不持有 Activity、Fragment、Dialog、View 或 Context。
- 页面重建后仍需存在的数据属于 UiState。
- 导航、Toast、Dialog 等一次性动作必须在展示后消费，避免旋转后重复执行。
- 异步请求开始和结束时必须正确更新 loading 状态。

6.3 Dialog

- 继承 `BaseDialog<对应的Binding>`。
- 使用 `newInstance(...)` 和 Fragment arguments 传参。
- 不使用自定义构造参数，以保证 Android 能恢复 DialogFragment。
- 对话框已经显示时，应通过稳定 tag 防止重复创建。

6.4 图片

- 使用 `GlideImageLoader`，不在页面重复编写 Glide 请求链。
- 网络图片必须提供 placeholder 和 error drawable。
- 头像使用 `loadCircle`，普通菜品图使用 `load`。
- 不把 Bitmap 或 Drawable 长期保存在 ViewModel 或单例中。

7. Repository 与 Firebase 接入

上层只能依赖 `data/repository` 中的接口。Firebase 实现建议使用下列结构：

```
data/repository/firebase/
├── FirebaseAuthRepository.java
├── FirebaseBookingRepository.java
├── FirebaseMenuRepository.java
├── FirebaseOrderRepository.java
└── FirebaseTableRepository.java
```

实现类负责把 Firebase 的回调、异常和快照转换成项目自己的实体与 `RepositoryCallback<T>`。Firebase 的 `Task`、`DataSnapshot` 和 `DatabaseReference` 不应暴露给 Activity。

7.1 登录流程

```
LoginActivity 提交原始输入
  → LoginViewModel 调用 LoginValidator
  → 校验失败：发布字段错误
  → 校验成功：设置 loading=true，调用 AuthRepository.signIn
  → Repository 返回 User 或错误
  → ViewModel 设置 loading=false，发布结果
  → Activity 消费一次性提示或导航事件
```

网络错误应转换成用户可理解的字符串资源；技术异常可在调试环境记录，但不得 记录密码、Token 或完整个人资料。

8. Realtime Database 建议结构

第一版可采用以下顶层节点：

```
users/{uid}
menuCategories/{categoryId}
menuItems/{menuItemId}
tables/{tableId}
bookings/{bookingId}
orders/{orderId}
orderItems/{orderItemId}
```

关系通过实体中的 ID 字段建立。需要按顾客、状态或分类查询的节点，应在安全规则 中添加 `.indexOn`。只有在真实查询出现性能或权限问题时，才增加按用户或状态的 反规范化索引节点；同一业务写入涉及多个节点时使用原子 multi-location update。

8.1 数据约定

- 金额保存为 `long` 类型的 cents，例如 `$35.99` 保存为 `3599`。
- 时间保存为 `long` 类型的 Unix epoch milliseconds。
- 状态只使用 `domain/constant` 中定义的字符串。
- UID 来自 Firebase Authentication，不自行生成用户 ID。
- 订单明细保留下单时的菜品名称和价格快照。
- 删除被历史订单引用的菜品时优先设为不可用，不直接物理删除。

8.2 权限方向

数据	Customer	Staff	Manager
自己的用户资料	读/有限修改	读/有限修改	读/有限修改
菜单	读	读	新增/修改/下架
自己的预订	新增/读/取消	-	读
全部预订	-	读/更新状态	读/更新状态
自己的订单	新增/读	-	读
全部订单	-	读/更新状态	读/更新状态
餐桌	读可用信息	读/更新状态	新增/修改/停用
员工账号	-	-	创建/启用/停用

最终权限必须写入 Firebase Security Rules。客户端隐藏按钮只改善体验，不能代替 服务端规则。

9. 错误、加载与离线状态

每个读取数据的页面至少设计以下状态：

- `loading`：首次加载或提交中，阻止重复提交。
- `content`：已有可展示数据。
- `empty`：请求成功但没有数据，给出下一步操作。
- `fieldError`：可定位到输入框的错误。
- `generalError`：网络、权限或未知错误，并提供重试入口。

异步回调返回后应检查页面生命周期；Activity 只观察 LiveData，不让 Repository 直接操作页面。若以后加入离线缓存，应先明确数据新旧标识和冲突解决策略。

10. 测试策略

10.1 单元测试

- Validator：空值、边界、格式和成功路径。
- ViewModel：初始状态、loading 顺序、成功、错误和一次性事件消费。
- 实体：无参构造、全字段构造、所有 getter/setter。
- Repository 契约：方法签名不暴露 Firebase 类型。

10.2 Robolectric / UI 集成测试

- ViewBinding 正确加载。
- 输入错误显示在正确字段。
- 第一个错误字段获得焦点。
- 成功和注册提示 Dialog 正确显示。
- Activity 重建后不会重复显示一次性 Dialog。

- 最低 API 24 与目标 API 35 都应覆盖关键页面。

10.3 Firebase 测试

接入 Firebase 后使用 Emulator Suite 验证：

- 未认证用户不能读取业务数据。
- Customer 不能修改他人数据或管理数据。
- Staff 只能执行被授权的状态变更。
- Manager 权限符合需求。
- 错误字段、越权状态和非法金额会被规则拒绝。

11. 构建与发布

常用命令：

```
./gradlew testDebugUnitTest
./gradlew lintDebug
./gradlew assembleDebug
```

提交前建议执行：

```
./gradlew clean testDebugUnitTest lintDebug assembleDebug
```

Release 前还需完成签名配置、版本号更新、Firebase 正式环境切换和真机冒烟测试。 签名文件、密钥密码、`google-services.json` 和本机 `local.properties` 不得提交。

12. 架构变更原则

- 修改实体字段、状态值、Repository 接口或数据库路径前必须与全组确认。
- 破坏兼容性的数据库修改需要迁移方案，不能只更新客户端代码。
- 引入新库前说明用途、体积、维护状态和现有能力为何不足。
- 新抽象至少应有两个真实使用场景；否则先保持简单实现。
- 架构发生变化时，同一 Pull Request 必须同步更新本文档和相关测试。

13. 新功能完成检查表

- □ 文件位于正确包中，命名符合团队规则。
- □ Activity/Dialog 全部使用 ViewBinding。
- □ 业务逻辑位于 ViewModel、Validator 或 Repository。
- □ Firebase 类型没有泄漏到 UI 层。
- □ loading、空状态、字段错误和通用错误均有处理。
- □ 用户文字位于 `strings.xml`，图片有替代说明。
- □ 不记录密码、Token 或敏感个人资料。
- □ 新实体满足 Firebase 构造和字段注释要求。
- □ 新行为有测试，完整测试和构建通过。
- □ 文档、Firebase 规则和索引在需要时同步更新。

PART 02

Just Eat 团队开发规则

本文档是项目的团队协作规则。所有成员提交到 `main` 前都应满足这些要求。

1. 基本原则

1. 只实现当前任务范围，避免在一个 Pull Request 中混入无关重构。
2. UI、业务状态和数据访问分层；任何层都不能绕过既定边界。
3. 新行为必须可验证，错误路径与成功路径同等重要。
4. Firebase 权限采用最小权限原则，不能依赖客户端界面隐藏来保护数据。
5. 密码、Token、密钥和配置凭据不得写入代码、日志或 Git 历史。

2. 分支与提交

2.1 分支命名

从最新 `main` 创建短生命周期分支：

```
feature/customer-menu
feature/table-booking
fix/login-validation
docs/firebase-rules
test/order-viewmodel
```

不允许长期在个人分支积累多个无关功能。合并前先同步 `main` 并解决冲突。

2.2 Commit 信息

使用英文 Conventional Commit 格式：

```
feat: add customer menu list
fix: prevent duplicate booking submission
test: cover order status transitions
docs: update Firebase setup guide
refactor: extract booking validator
```

每个 commit 应保持可编译，并只表达一个主要目的。禁止使用 `update`、`changes`、 `final` 等无法说明内容的提交信息。

2.3 Pull Request

- `main` 只接收经过审查的完整功能或修复。
- PR 描述必须包含：目的、主要改动、测试方法、截图（涉及 UI 时）和已知限制。
- 至少一名其他成员完成审查后再合并。
- 作者必须处理 Critical 和 Important 问题；Minor 问题可记录后续任务。
- 禁止通过 force push 或重写共享 `main` 历史解决普通冲突。

3. Java 规则

- 生产代码使用 Java 17，不在项目中混入 Kotlin。
- 一个 `.java` 文件只放一个 public 顶层类。
- 类名使用 PascalCase，方法和变量使用 camelCase，常量使用 UPPER_SNAKE_CASE。
- 方法保持单一职责；复杂分支优先拆成可命名、可测试的小方法。
- 对外 API、基类、实体字段和非显而易见规则使用中文 Javadoc/注释。
- 不保留未使用代码、注释掉的大段代码或无负责人 TODO。
- 不捕获异常后静默忽略；应转换为明确的 UI 状态或 Repository 错误。
- 不在 Activity、ViewModel 或 Repository 中记录密码和 Token。

4. Android UI 规则

- Activity 必须继承 `BaseActivity<VB>` 并使用 ViewBinding。
- DialogFragment 必须继承 `BaseDialog<VB>` 并使用 Fragment arguments 传参。
- 生产页面禁止使用 `findViewById`；测试代码可以用于取得 Android content root。
- Activity 只绑定操作和渲染状态，不直接调用 Firebase 或包含业务规则。
- 所有用户可见文字放入 `strings.xml`，禁止硬编码在 Java 或布局中。
- 可点击控件目标尺寸至少为 48dp，并提供明确的 enabled/loading 状态。
- 有意义的图片必须提供 `contentDescription`；纯装饰图片明确设为无障碍忽略。
- 表单错误显示在对应输入框，提交失败后聚焦第一个错误字段。
- 一次性 Dialog、Toast 和导航事件展示后必须消费，并使用稳定 tag 去重。
- 布局应支持小屏、键盘弹出和横竖屏；长表单使用可滚动容器。

5. ViewModel 与状态规则

- ViewModel 不持有 Activity、Fragment、View、ViewBinding、Dialog 或 Context。
- UI 使用只读 `LiveData` 观察状态，MutableLiveData 保持 private。
- UiState 应不可变，并能完整描述当前页面需要渲染的状态。
- 每次异步操作都必须正确结束 loading，包括失败和提前返回路径。
- Repository 回调先转换成 ViewModel 状态，再由 Activity 渲染。
- 旋转或重建后不应重复提交网络请求或重复弹出已经消费的提示。

6. Repository 与 Firebase 规则

- Activity 和 Dialog 禁止直接引用 `FirebaseAuth`、`DatabaseReference` 或 `DataSnapshot`。
- Firebase 实现只放在 `data/repository/firebase`。
- 上层依赖现有 Repository 接口；确需修改接口时同步更新实现、测试和文档。
- 数据读取和写入必须处理成功、权限失败、网络失败和数据格式错误。

- 多节点相关写入使用原子 multi-location update，避免半完成状态。
- 上线前使用 Firebase Emulator Suite 测试安全规则。
- 数据库禁止开放全局 `.read: true` 或 `.write: true`。
- 角色权限必须由 Security Rules 验证，不能信任客户端传入的 role。

7. 实体与数据规则

每个 Firebase 实体必须同时具备：

- public 无参构造方法。
- 包含全部字段的 public 构造方法。
- 每个字段的 getter 和 setter。
- 类、字段、构造方法和特殊约束的中文注释。

统一数据约定：

- 金额使用 `long` cents，禁止使用 `float` 或 `double` 保存价格。
- 时间使用 `long` Unix epoch milliseconds。
- ID 使用 `String`，并保持 Firebase 路径键与实体 ID 一致。
- 状态值来自 `domain/constant`，禁止在页面中散落魔法字符串。
- 历史订单保存菜品名称和价格快照，不能依赖当前菜单价格还原历史。
- 实体不包含密码、认证 Token、Android View 或 Firebase SDK 对象。

8. 测试规则

- 新功能或修复先添加能失败的测试，再编写生产实现。
- Validator 必须覆盖 null、空白、边界和成功路径。
- ViewModel 必须覆盖初始、loading、成功、错误和一次性事件。
- 新实体必须测试无参构造、全字段构造及全部 getter/setter。
- UI 关键流程使用 Robolectric 或设备测试，并覆盖最低支持 API。
- 修复崩溃时保留可防止问题复发的测试或可重复冒烟步骤。

提交前至少运行：

```
./gradlew testDebugUnitTest
./gradlew lintDebug
./gradlew assembleDebug
```

9. 安全与隐私红线

以下内容禁止提交或记录：

- google-services.json
- keystore、签名密码和服务账号密钥
- Firebase/API Token
- 用户密码

- 真实用户的完整个人资料或生产数据库导出

发现凭据误提交时，应立即停止共享、撤销或轮换凭据，并清理 Git 历史；只删除 最新文件并不能使已经进入历史的密钥失效。

10. 审查检查表

审查者至少检查：

- □ 功能符合 issue/任务范围。
- □ 分层和依赖方向正确。
- □ 生命周期与配置变更处理正确。
- □ Firebase 权限和错误路径安全。
- □ 没有密码、Token 或凭据泄漏。
- □ 实体和数据库字段保持兼容。
- □ UI 文字、无障碍和小屏行为合理。
- □ 测试验证真实行为，而不是只验证 mock 调用。
- □ 文档与代码同步。

11. Definition of Done

一个任务只有同时满足以下条件才算完成：

1. 验收条件全部实现，没有越出任务范围。
2. 代码位于正确分层，没有已知 Critical 或 Important 审查问题。
3. 新行为有测试，完整测试为零失败。
4. Lint 没有新增错误；新增警告有明确原因。
5. Debug APK 可构建，相关页面完成真机或模拟器冒烟测试。
6. 没有提交敏感文件，日志不包含敏感信息。
7. README、开发指南、Firebase 说明或规则已按需更新。
8. PR 已由至少一名其他成员审查并合并。

12. 争议处理

遇到实体字段、数据库路径、状态名称、Repository 接口或架构方向分歧时，不要让 每个分支各自实现。先在 issue/小组记录中写明选项和影响，由全组确认后更新文档， 再继续编码。已写入本规范的规则优先于个人习惯；确需改变时通过独立 PR 修改规范。

文档维护

当架构、依赖、数据库路径、角色权限、交付流程或团队共识发生变化时，应在同一个 Pull Request 中更新 Markdown 原文，并重新生成本 PDF。Markdown 是便于审查的原始文档，PDF 是便于阅读和提交的发布版本。